

Water Distribution and Consumption Management System

A Menu-Driven Python Application

Data Structure: Dictionary of Sets

Pseudocode

```
BEGIN WaterManagementSystem
```

```
  DECLARE water_data AS empty DICTIONARY of SETS    // key = household/building ID
                                                    // value = SET of record TUPLES
  DECLARE THRESHOLDS AS TUPLE (100, 300)           // (low_bound, moderate_bound)
```

```
  FUNCTION main():
    LOOP FOREVER
      DISPLAY menu (options 1 to 12)
      READ choice
      IF choice == 12 THEN
        PRINT "Exiting..."
        BREAK
      ELSE
        CALL the function mapped to choice
      END IF
    END LOOP
  END FUNCTION
```

```
  FUNCTION add_record():
    READ household_id, zone, supplied, consumed, date
    VALIDATE each field using string/number checks
    IF validation fails THEN PRINT error AND RETURN
    record = TUPLE(zone, supplied, consumed, date)  // tuples are hashable
    IF household_id EXISTS in water_data THEN
      IF record ALREADY IN water_data[household_id] THEN
        PRINT "duplicate ignored" AND RETURN
      END IF
      ADD record to water_data[household_id]        // SET.add()
    ELSE
      water_data[household_id] = SET containing record
    END IF
  END FUNCTION
```

```
  FUNCTION display_records():
    IF water_data is empty THEN PRINT "No records" AND RETURN
    FOR EACH household_id, records IN water_data
      FOR EACH (zone, supplied, consumed, date) IN records  // set iteration
        PRINT household_id, zone, supplied, consumed, date, classify(consumed)
      END FOR
    END FOR
  END FUNCTION
```

```
  FUNCTION search_records():
    READ search_type (ID / Zone / Date)
    READ search_value
    FOR EACH household_id, records IN water_data
      FOR EACH record IN records
        IF record MATCHES search_value on search_type THEN
          PRINT matching record
        END IF
      END FOR
    END FOR
  END FUNCTION
```

```
  FUNCTION classify(consumed):
    IF consumed <= THRESHOLDS.low THEN RETURN "Low"
    ELSE IF consumed <= THRESHOLDS.moderate THEN RETURN "Moderate"
    ELSE RETURN "High"
    END IF
  END FUNCTION
```

```

FUNCTION water_analysis():
    total_supplied = SUM of supplied over all records in all sets
    total_consumed = SUM of consumed over all records in all sets
    remaining = total_supplied - total_consumed
    average = MEAN of consumed values
    percentage = (total_consumed / total_supplied) * 100
    PRINT total_supplied, total_consumed, remaining, average, percentage
END FUNCTION

FUNCTION identify_excessive_consumption():
    READ limit
    FOR EACH household_id, records IN water_data
        FOR EACH record IN records
            IF record.consumed > limit THEN
                PRINT household_id, record AS "EXCESSIVE"
            END IF
        END FOR
    END FOR
END FUNCTION

FUNCTION zone_wise_analysis():
    DECLARE zone_readings AS empty DICTIONARY of SETS
    FOR EACH record IN all records
        ADD (consumed, date) to zone_readings[record.zone]    // SET, dedupes
    END FOR
    FOR EACH zone, readings IN zone_readings
        PRINT zone, SUM(readings), MEAN(readings), COUNT(readings)
    END FOR
END FUNCTION

FUNCTION unique_data_analysis():
    unique_households = SET of all household_id keys
    unique_zones = SET of all record.zone values
    unique_categories = SET of classify(record.consumed) for all records
    PRINT unique_households, unique_zones, unique_categories
END FUNCTION

FUNCTION save_records_to_file():
    CONVERT each SET of TUPLES to a LIST of LISTS    // JSON has no set/tuple type
    WRITE converted water_data to DATA_FILE in JSON format
END FUNCTION

FUNCTION load_records_from_file():
    READ raw data from DATA_FILE (JSON format)
    REBUILD water_data: each LIST of LISTS -> SET of TUPLES
END FUNCTION

FUNCTION generate_report():
    COMPUTE totals as in water_analysis()
    WRITE totals AND every record with its category to REPORT_FILE
END FUNCTION

END WaterManagementSystem

```

Source Code

water_management_system_dos.py

```

"""
=====
WATER DISTRIBUTION AND CONSUMPTION MANAGEMENT SYSTEM
=====
A menu-driven Python application to manage water distribution and
consumption activities in a residential / community area.

Primary Data Structure : DICTIONARY OF SETS
    water_data = {
        "H001": { (zone, water_supplied, water_consumed, date), ... },
        "H002": { (zone, water_supplied, water_consumed, date), ... },
        ...
    }
    Each record is stored as a TUPLE (immutable, hashable) so it can live
    inside a SET. A set automatically prevents an identical record being
    stored twice for the same household, and set operations (union,
    membership testing) are used throughout the analysis functions.

Concepts demonstrated : String operations & functions, Tuples, Sets,
                        Dictionaries, File handling, Modules.
Course Outcomes       : CO3, CO4
SDG Mapping           : SDG 6, SDG 11, SDG 12, SDG 13
=====
"""

import os
import json
import statistics
from datetime import datetime

# -----
# GLOBAL DATA STRUCTURE
# -----
# water_data -> DICTIONARY OF SETS
#             Key   : household/building ID (string)
#             Value : SET of records, where each record is a TUPLE
#                   (zone, supplied, consumed, date)
water_data = {}

# Consumption classification thresholds (in litres), stored as a TUPLE
# because these are fixed, ordered boundary values.
THRESHOLDS = (100, 300)          # (low_upper_bound, moderate_upper_bound)

DATA_FILE = "water_records.txt"
REPORT_FILE = "water_analysis_report.txt"

# -----
# STRING VALIDATION HELPER FUNCTIONS
# -----
def clean_text(value):
    """Trim whitespace for consistent storage/searching."""
    return value.strip()

def is_valid_id(household_id):
    """A valid household/building ID: non-empty, letters/digits/hyphen only."""
    household_id = household_id.strip()
    if household_id == "":
        return False
    return all(ch.isalnum() or ch == "-" for ch in household_id)

```

```

def is_valid_number(value):
    """Check whether a string represents a valid non-negative number."""
    value = value.strip()
    if value == "":
        return False
    if value.count(".") > 1:
        return False
    stripped = value.replace(".", "", 1)
    return stripped.isdigit()

def is_valid_date(value):
    """Validate a date string in DD-MM-YYYY format."""
    value = value.strip()
    parts = value.split("-")
    if len(parts) != 3:
        return False
    day, month, year = parts
    if not (day.isdigit() and month.isdigit() and year.isdigit()):
        return False
    try:
        datetime(int(year), int(month), int(day))
        return True
    except ValueError:
        return False

# -----
# 1. ADD WATER DISTRIBUTION RECORD
# -----
def add_record():
    """Prompt for record details, validate them, and store the record as a
    TUPLE inside the SET belonging to this household (Dictionary of Sets)."""
    print("\n--- Add Water Distribution Record ---")

    household_id = clean_text(input("Enter Household/Building ID: ")).upper()
    if not is_valid_id(household_id):
        print("Invalid ID. Use letters, digits and hyphens only.")
        return

    zone = clean_text(input("Enter Area/Zone: ")).title()
    if zone == "":
        print("Zone cannot be empty.")
        return

    supplied = input("Enter Water Supplied (litres): ")
    if not is_valid_number(supplied):
        print("Invalid number for water supplied.")
        return
    supplied = float(supplied)

    consumed = input("Enter Water Consumption (litres): ")
    if not is_valid_number(consumed):
        print("Invalid number for water consumption.")
        return
    consumed = float(consumed)

    if consumed > supplied:
        print("Warning: consumption exceeds supplied quantity for this entry.")

    date_str = clean_text(input("Enter Date (DD-MM-YYYY): "))
    if not is_valid_date(date_str):
        print("Invalid date format.")
        return

```

```

# record stored as a TUPLE -> (zone, supplied, consumed, date)
record = (zone, supplied, consumed, date_str)

# Dictionary of Sets: add to household's set, or create a new set
if household_id in water_data:
    if record in water_data[household_id]:
        print("This exact record already exists for this household – duplicate ignored.")
        return
    water_data[household_id].add(record)
else:
    water_data[household_id] = {record}

print(f"Record added successfully for household '{household_id}'.")

# -----
# 2. DISPLAY RECORDS
# -----
def display_records():
    """Display every stored water distribution/consumption record."""
    print("\n--- All Water Distribution & Consumption Records ---")
    if not water_data:
        print("No records available.")
        return

    header =
f"{'ID':<10}{'Zone':<12}{'Supplied(L)':<14}{'Consumed(L)':<14}{'Date':<12}{'Category':<10}"
    print(header)
    print("-" * len(header))
    # Sets are unordered, so records are sorted by date purely for readable
    #, repeatable console output.
    for household_id, records in water_data.items():
        for zone, supplied, consumed, date_str in sorted(records, key=lambda r: r[3]):
            category = classify_single(consumed)

print(f"{household_id:<10}{zone:<12}{supplied:<14.1f}{consumed:<14.1f}{date_str:<12}{category:<10}")

# -----
# 3. SEARCH RECORDS
# -----
def search_records():
    """Search records by household ID, zone, or date."""
    print("\n--- Search Records ---")
    print("1. Search by Household/Building ID")
    print("2. Search by Zone")
    print("3. Search by Date")
    choice = input("Enter choice: ").strip()

    found = False
    if choice == "1":
        key = clean_text(input("Enter Household/Building ID: ")).upper()
        if key in water_data:
            for zone, supplied, consumed, date_str in water_data[key]:
                print(f"{key} -> Zone:{zone} Supplied:{supplied} Consumed:{consumed} Date:{date_str}")
                found = True
    elif choice == "2":
        key = clean_text(input("Enter Zone: ")).title()
        for household_id, records in water_data.items():
            for zone, supplied, consumed, date_str in records:
                if zone == key:
                    print(f"{household_id} -> Zone:{zone} Supplied:{supplied} Consumed:{consumed}
Date:{date_str}")
                    found = True
    elif choice == "3":

```

```

        key = clean_text(input("Enter Date (DD-MM-YYYY): "))
        for household_id, records in water_data.items():
            for zone, supplied, consumed, date_str in records:
                if date_str == key:
                    print(f"{household_id} -> Zone:{zone} Supplied:{supplied} Consumed:{consumed}
Date:{date_str}")
                    found = True
            else:
                print("Invalid choice.")
                return

        if not found:
            print("No matching records found.")

# -----
# 4. CONSUMPTION CLASSIFICATION
# -----
def classify_single(consumed):
    """Classify one consumption value as Low / Moderate / High."""
    low_bound, moderate_bound = THRESHOLDS
    if consumed <= low_bound:
        return "Low"
    elif consumed <= moderate_bound:
        return "Moderate"
    else:
        return "High"

def consumption_classification():
    """Classify and display every household's records into categories."""
    print("\n--- Consumption Classification ---")
    if not water_data:
        print("No records available.")
        return

    for household_id, records in water_data.items():
        for zone, supplied, consumed, date_str in sorted(records, key=lambda r: r[3]):
            category = classify_single(consumed)
            print(f"{household_id} | {date_str} | Consumption: {consumed} L -> {category}")

# -----
# 5. WATER ANALYSIS
# -----
def water_analysis():
    """Calculate total supplied, total consumed, remaining water, average
consumption and consumption percentage across all records."""
    print("\n--- Water Analysis ---")
    if not water_data:
        print("No records available.")
        return

    all_supplied = []
    all_consumed = []
    for records in water_data.values():
        for zone, supplied, consumed, date_str in records:
            all_supplied.append(supplied)
            all_consumed.append(consumed)

    total_supplied = sum(all_supplied)
    total_consumed = sum(all_consumed)
    remaining = total_supplied - total_consumed
    average_consumption = statistics.mean(all_consumed) if all_consumed else 0
    consumption_pct = (total_consumed / total_supplied * 100) if total_supplied else 0

```

```

print(f"Total Water Supplied      : {total_supplied:.2f} L")
print(f"Total Water Consumed     : {total_consumed:.2f} L")
print(f"Remaining Water          : {remaining:.2f} L")
print(f"Average Consumption      : {average_consumption:.2f} L")
print(f"Consumption Percentage    : {consumption_pct:.2f} %")

return total_supplied, total_consumed, remaining, average_consumption, consumption_pct

# -----
# 6. IDENTIFY EXCESSIVE CONSUMPTION
# -----
def identify_excessive_consumption():
    """Detect households/areas whose consumption exceeds a user-specified
    limit."""
    print("\n--- Identify Excessive Consumption ---")
    if not water_data:
        print("No records available.")
        return

    limit_str = input("Enter consumption limit (litres): ")
    if not is_valid_number(limit_str):
        print("Invalid limit.")
        return
    limit = float(limit_str)

    exceeded = False
    for household_id, records in water_data.items():
        for zone, supplied, consumed, date_str in records:
            if consumed > limit:
                print(f"EXCESSIVE -> {household_id} | Zone:{zone} | Consumed:{consumed} L |
Date:{date_str}")
                exceeded = True

    if not exceeded:
        print(f"No households exceeded the limit of {limit} L.")

# -----
# 7. ZONE-WISE ANALYSIS
# -----
def zone_wise_analysis():
    """Compare total and average water consumption among different zones.
    Uses a dictionary of sets internally: zone -> set of consumption
    readings, so an identical repeated reading in the same zone is only
    counted once."""
    print("\n--- Zone-wise Analysis ---")
    if not water_data:
        print("No records available.")
        return

    zone_readings = {} # dictionary of SETS: zone -> {consumed, ...}
    for records in water_data.values():
        for zone, supplied, consumed, date_str in records:
            zone_readings.setdefault(zone, set()).add((consumed, date_str))

    print(f"'Zone':<15{'Total (L)':<14{'Average (L)':<14{'Entries':<10}")
    print("-" * 53)
    for zone, readings in zone_readings.items():
        values = [r[0] for r in readings]
        total = sum(values)
        avg = statistics.mean(values)
        print(f"{zone:<15}{total:<14.2f}{avg:<14.2f}{len(values):<10}")

# -----

```



```

# 8. UNIQUE DATA ANALYSIS (SETS)
# -----
def unique_data_analysis():
    """Use sets to identify unique zones, unique household IDs and unique
    consumption categories present in the data."""
    print("\n--- Unique Data Analysis (using sets) ---")
    if not water_data:
        print("No records available.")
        return

    unique_households = set(water_data.keys())
    unique_zones = set()
    unique_categories = set()

    for records in water_data.values():
        for zone, supplied, consumed, date_str in records:
            unique_zones.add(zone)
            unique_categories.add(classify_single(consumed))

    print(f"Unique Households/Buildings : {sorted(unique_households)}")
    print(f"Unique Zones : {sorted(unique_zones)}")
    print(f"Unique Consumption Categories: {sorted(unique_categories)}")

# -----
# 9. FILE HANDLING
# -----
def save_records_to_file():
    """Persist the Dictionary of Sets to a text file using JSON encoding.
    JSON has no native set/tuple type, so each set is converted to a
    sorted list of lists before writing, and rebuilt as a dictionary of
    sets of tuples on load."""
    try:
        serialisable = {
            household_id: sorted([list(rec) for rec in records], key=lambda r: r[3])
            for household_id, records in water_data.items()
        }
        with open(DATA_FILE, "w") as f:
            json.dump(serialisable, f, indent=4)
        print(f"Records saved to '{DATA_FILE}' successfully.")
    except IOError as e:
        print(f"Error saving file: {e}")

def load_records_from_file():
    """Retrieve previously stored records from the data file and rebuild
    the Dictionary of Sets (lists -> tuples -> set)."""
    global water_data
    if not os.path.exists(DATA_FILE):
        print("No saved file found.")
        return
    try:
        with open(DATA_FILE, "r") as f:
            raw = json.load(f)
        water_data = {
            household_id: {tuple(rec) for rec in records}
            for household_id, records in raw.items()
        }
        print(f"Records loaded from '{DATA_FILE}' successfully.")
    except (IOError, json.JSONDecodeError) as e:
        print(f"Error loading file: {e}")

def generate_report():
    """Generate an analytical text report summarising the whole data set
    and write it to REPORT_FILE."""

```

```

if not water_data:
    print("No records available to generate a report.")
    return

lines = []
lines.append("WATER DISTRIBUTION & CONSUMPTION ANALYSIS REPORT")
lines.append("=" * 55)
lines.append(f"Generated on: {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}")
lines.append("")

total_supplied = total_consumed = 0
all_consumed = []
for records in water_data.values():
    for zone, supplied, consumed, date_str in records:
        total_supplied += supplied
        total_consumed += consumed
        all_consumed.append(consumed)

remaining = total_supplied - total_consumed
avg = statistics.mean(all_consumed) if all_consumed else 0
pct = (total_consumed / total_supplied * 100) if total_supplied else 0

lines.append(f"Total Households/Buildings : {len(water_data)}")
lines.append(f"Total Water Supplied      : {total_supplied:.2f} L")
lines.append(f"Total Water Consumed       : {total_consumed:.2f} L")
lines.append(f"Remaining Water            : {remaining:.2f} L")
lines.append(f"Average Consumption        : {avg:.2f} L")
lines.append(f"Consumption Percentage     : {pct:.2f} %")
lines.append("")
lines.append("Household-wise Details")
lines.append("-" * 55)
for household_id, records in water_data.items():
    for zone, supplied, consumed, date_str in sorted(records, key=lambda r: r[3]):
        category = classify_single(consumed)
        lines.append(f"{household_id} | Zone:{zone} | Supplied:{supplied} | "
                    f"Consumed:{consumed} | Date:{date_str} | {category}")

try:
    with open(REPORT_FILE, "w") as f:
        f.write("\n".join(lines))
    print(f"Report generated: '{REPORT_FILE}'")
except IOError as e:
    print(f"Error writing report: {e}")

# -----
# 10. MENU-DRIVEN INTERFACE
# -----
def display_menu():
    print("\n" + "=" * 55)
    print(" WATER DISTRIBUTION AND CONSUMPTION MANAGEMENT SYSTEM")
    print("=" * 55)
    print("1. Add Water Distribution Record")
    print("2. Display Records")
    print("3. Search Records")
    print("4. Consumption Classification")
    print("5. Water Analysis")
    print("6. Identify Excessive Consumption")
    print("7. Zone-wise Analysis")
    print("8. Unique Data Analysis")
    print("9. Save Records to File")
    print("10. Load Records from File")
    print("11. Generate Analytical Report")
    print("12. Exit")

```

```
def main():
    menu_actions = {
        "1": add_record,
        "2": display_records,
        "3": search_records,
        "4": consumption_classification,
        "5": water_analysis,
        "6": identify_excessive_consumption,
        "7": zone_wise_analysis,
        "8": unique_data_analysis,
        "9": save_records_to_file,
        "10": load_records_from_file,
        "11": generate_report,
    }

    while True:
        display_menu()
        choice = input("Enter your choice (1-12): ").strip()
        if choice == "12":
            print("Exiting... Thank you for using the Water Management System.")
            break
        action = menu_actions.get(choice)
        if action:
            action()
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Output Screenshots

```

Terminal - python3 water_management_system.py

$ python3 water_management_system.py
=====
WATER DISTRIBUTION AND CONSUMPTION MANAGEMENT SYSTEM
=====
1. Add Water Distribution Record
2. Display Records
3. Search Records
4. Consumption Classification
5. Water Analysis
6. Identify Excessive Consumption
7. Zone-wise Analysis
8. Unique Data Analysis
9. Save Records to File
10. Load Records from File
11. Generate Analytical Report
12. Exit

```

SCREENSHOT 1:

```

Terminal - Option 1: Add Water Distribution Record

Enter your choice (1-12): 1
Enter Household/Building ID: H004
Enter Area/Zone: West
Enter Water Supplied (litres): 600
Enter Water Consumption (litres): 150
Enter Date (DD-MM-YYYY): 10-08-2026
--- Add Water Distribution Record ---
Record added successfully for household 'H004'.

```

SCREENSHOT 2:

```

Terminal - Option 2: Display Records

Enter your choice (1-12): 2
--- All Water Distribution & Consumption Records ---

```

ID	Zone	Supplied(L)	Consumed(L)	Date	Category
H001	North	500.0	120.0	01-08-2026	Moderate
H001	North	500.0	480.0	05-08-2026	High
H002	South	400.0	90.0	01-08-2026	Low
B010	East	1000.0	950.0	02-08-2026	High
H004	West	600.0	150.0	10-08-2026	Moderate

SCREENSHOT 3:

```

Terminal – Option 3: Search Records

Enter your choice (1-12): 3
Enter choice: 1
Enter Household/Building ID: H001
--- Search Records ---
1. Search by Household/Building ID
2. Search by Zone
3. Search by Date
H001 -> Zone:North Supplied:500.0 Consumed:480.0 Date:05-08-2026
H001 -> Zone:North Supplied:500.0 Consumed:120.0 Date:01-08-2026

```

SCREENSHOT 4:

```

Terminal – Option 4: Consumption Classification

Enter your choice (1-12): 4
--- Consumption Classification ---
H001 | 01-08-2026 | Consumption: 120.0 L -> Moderate
H001 | 05-08-2026 | Consumption: 480.0 L -> High
H002 | 01-08-2026 | Consumption: 90.0 L -> Low
B010 | 02-08-2026 | Consumption: 950.0 L -> High
H004 | 10-08-2026 | Consumption: 150.0 L -> Moderate

```

SCREENSHOT 5:

```

Terminal – Option 5: Water Analysis

Enter your choice (1-12): 5
--- Water Analysis ---
Total Water Supplied      : 3000.00 L
Total Water Consumed      : 1790.00 L
Remaining Water           : 1210.00 L
Average Consumption       : 358.00 L
Consumption Percentage    : 59.67 %

```

SCREENSHOT 6:

```

Terminal – Option 6: Identify Excessive Consumption

Enter your choice (1-12): 6
Enter consumption limit (litres): 300
--- Identify Excessive Consumption ---
EXCESSIVE -> H001 | Zone:North | Consumed:480.0 L | Date:05-08-2026
EXCESSIVE -> B010 | Zone:East | Consumed:950.0 L | Date:02-08-2026

```

SCREENSHOT 7:

```

Terminal – Option 7: Zone-wise Analysis

Enter your choice (1-12): 7
--- Zone-wise Analysis ---
Zone          Total (L)    Average (L)    Entries
-----
North         600.00         300.00         2
South         90.00          90.00          1
East          950.00         950.00         1
West          150.00         150.00         1

```

SCREENSHOT 8:

```
Terminal – Option 8: Unique Data Analysis

Enter your choice (1-12): 8
--- Unique Data Analysis (using sets) ---
Unique Households/Buildings : ['B010', 'H001', 'H002', 'H004']
Unique Zones                : ['East', 'North', 'South', 'West']
Unique Consumption Categories: ['High', 'Low', 'Moderate']
```

SCREENSHOT 9:

```
Terminal – Option 9: Save Records to File

Enter your choice (1-12): 9
Records saved to 'water_records.txt' successfully.
```

SCREENSHOT 10:

```
Terminal – Option 10: Load Records from File

Enter your choice (1-12): 10
Records loaded from 'water_records.txt' successfully.
```

SCREENSHOT 11:

```
Terminal – Option 11: Generate Analytical Report

Enter your choice (1-12): 11
Report generated: 'water_analysis_report.txt'
```